Objectifs CSP : Incremental, Online, Hybrid-n_j

1 Notation commune

Pour un ensemble de jobs $\mathcal{J} = \{1, \dots, n\}$ à (re)placer dans un solve donné, et pour chaque job $i \in \mathcal{J}$:

- end_i = instant de fin de la dernière tâche de i (variable de décision, dépend du placement choisi sur les nœuds) ;
- e_i = `timelapsed`, le temps déjà écoulé pour i à travers les replans précédents (constante, connue avant le solve) ;
- $\text{flow}_i = \text{end}_i + e_i$ — le *flow time* du job i, intégrant le temps d'attente déjà subi, pas seulement le temps d'exécution de ce solve.

Trois objectifs scalaires sont construits à partir de $\{\text{flow}_i\}_{i \in \mathcal{J}}$:

$$\text{(sum)} \qquad F_{\text{sum}} = \sum_{i \in \mathcal{J}} \text{flow}_i \tag{1}$$

$$\text{(max)} \qquad F_{\max} = \max_{i \in \mathcal{J}} \text{flow}_i \tag{2}$$

$$\text{(new job)} \qquad F_{\text{new}} = \text{flow}_{i^\star}, \quad i^\star = \arg\max_{i \in \mathcal{J}} \ \text{job_id}_i \tag{3}$$

($i^\star$ est conventionnellement le job au plus grand `job_id` du batch — par construction côté Python, c'est toujours le nouveau job qui vient d'arriver.)

Le choix entre ces trois formulations se fait via `objective_choice.txt` ($0 \to F_{\text{sum}}$, $1 \to F_{\max}$ par défaut, $2 \to F_{\text{new}}$), et le solve minimise la variable choisie :

$$\min \ F_{\text{objective_choice}}$$

1.1 Énergie (bi-objectif, Online/Hybrid uniquement)

Pour chaque candidat de transfert (j, i) (donnée i vers nœud j), un coût énergétique constant est précalculé :

$$c_{j,i} = \underbrace{\sigma}_{\text{énergie émetteur}} + \underbrace{\kappa_j \cdot d_{j,i}}_{\text{énergie récepteur} \times \text{durée transfert}} + \underbrace{\nu}_{\text{énergie réseau}}$$

où σ (`senderEnergy`), ν (`networkEnergy`) viennent de `energy_config.txt`, $\kappa_j =$ `node_energy_consumption` du nœud j, et $d_{j,i}$ la durée du transfert de i vers j. L'énergie totale est la somme pondérée par les transferts effectivement sélectionnés ($h_{j,i} \in \{0, 1\}$, 1 si le transfert (j, i) a lieu) :

$$E = \sum_j \sum_i h_{j,i} \cdot c_{j,i}$$

2 Incremental

Place uniquement le **nouveau job**, sans reconsidérer les jobs déjà en cours (ceux-ci restent figés tels quels — pas de variable de décision pour eux dans ce solve). L'objectif est toujours :

$$\min \; F_{\max}$$

appliqué au batch réduit à ce seul job (donc équivalent à $\min \text{flow}_{\text{new}}$ dans ce cas précis, puisque $\mathcal{J} = \{\text{new}\}$). Pas de choix d'objectif configurable, pas de volet énergie.

3 Online

Solve joint sur **tous** les jobs du batch (existants + nouveau) à chaque arrivée — aucun job n'est figé. Objectif sélectionnable via `objective_choice.txt` :

$$\min \; F_{\text{objective_choice}} \quad \text{(typiquement } F_{\max} \text{ par défaut)}$$

En mode bi-objectif (`MainOnlineMultiObj`, `multi_objective.txt=2`), recherche en deux phases (*epsilon-constraint*) :

$$\text{Phase 1:} \quad \min \; F_{\text{objective_choice}} \quad \rightarrow \quad F^\star \tag{4}$$

$$\text{Phase 2:} \quad \min \; E \quad \text{s.t.} \quad F_{\text{objective_choice}} \leq (1+\varepsilon)\, F^\star \tag{5}$$

où $\varepsilon =$ `epsilon_fraction` (10% par défaut). Le budget solveur total se répartit entre les deux phases (`epsilon_phase1_fraction`, 50/50 par défaut), et une borne absolue optionnelle (`epsilon_max_cap`) peut plafonner la phase 2 indépendamment du slack relatif.

4 Hybrid-n_j

Même mécanique qu'Online (solve joint), mais avec deux ajouts spécifiques :

1. **Objectif fixé à F_{new}** (`objective_choice=2`) — on optimise uniquement le flow time du nouveau job, jamais le batch entier.
2. **Plafond de dégradation par job** (optionnel, `flow_time_caps.txt`) — une borne dure par job existant :
$$\text{flow}_i \leq \text{cap}_i, \quad \text{cap}_i = \text{committed}_i \cdot (1+p)$$
où committed_i est l'estimation gloutonne pré-escalade du flow time de i (`_estimateJobFlowTime`, côté Python) et p le pourcentage de dégradation toléré (25% par défaut dans nos expériences). Empêche l'objectif étroit F_{new} d'améliorer le nouveau job au prix d'une dégradation arbitraire d'un job existant.
3. **Gel sélectif** (`frozen_jobs.txt` / `frozen_job_ids_override`) — un sous-ensemble $\mathcal{F} \subseteq \mathcal{J} \setminus \{\text{new}\}$ de jobs est confiné à ses nœuds de résidence actuels (aucun nouveau transfert autorisé pour ces jobs) ; c'est précisément le sous-ensemble $\mathcal{F}$ que les 8 heuristiques (et le modèle appris) cherchent à choisir.

Le même volet bi-objectif (phase 1 sur F_{new}, phase 2 sur E sous contrainte epsilon) s'applique identiquement, les plafonds de dégradation et le gel restant actifs dans les deux phases.